



Meet Web Push

Jun 7, 2022 by Brady Eidson



Websites have many reasons to notify their users of time-sensitive or high-priority events, even if the user does not currently have the site open. This feature is called Web Push, and is enabled by the W3C standards for [Push API](#), [Notifications API](#), and [Service Workers](#), all working together. WebKit now supports the relevant parts of those standards to enable Web Push.

Apple has made changes to macOS that deeply integrate with WebKit's support to provide a great user experience, and we're excited to announce that Web Push is supported in Safari 16 on macOS Ventura.

Keep an eye out for Web Push on iOS and iPadOS in 2023.

As long as you've coded your web application to the standards you will be able to reach Safari 16 users on macOS Ventura. You don't need to join the Apple Developer Program to send Web Push notifications.

If you exclude Safari through browser detection, now would be a great time to switch to feature detection, which lets you take advantage of new features as soon as they're supported. Additionally, if you tightly manage push end points on your server, be sure to allow URLs from any subdomain of `push.apple.com`.

All of this and more is covered in [Meet Web Push](#) (15 minute video) at WWDC22.

Standards overview

Most features of the web platform are described in a single web standard. Web Push is an exception, with multiple standards describing implementation requirements.

There are many resources on the web to help web application authors get up and running with these standards. But to further cover how WebKit's support works, it is useful to cover the web standards at a high level.

The Push API standard is the most directly relevant to start with. It describes the JavaScript interface that allows a website to register a push subscription. That subscription enables sending push messages to your user's browser using a push service.

The ServiceWorker API is extended to support these push messages. Once a push message is received from a domain, that domain's registered service worker script receives an event representing the push message.

The Notifications API is extended to allow service worker scripts to post a notification even without an open browser tab.

When a web application registers a push subscription, they promise that pushes will always be user visible. When the service worker handles a push message, it is required to use the Notifications API to display a user visible notification. Finally, when the user activates that notification, the service worker is sent an event representing the notification activation.

Power and privacy

Both the WebKit open source project and Apple treat privacy as a fundamental human right. As with other privileged features of the web platform, requesting a push subscription requires an explicit user gesture. It also requires you set the `userVisibleOnly` flag to true, and fulfill that promise by always showing a notification in response to a push message.

The Web Push API is not an invitation for silent background runtime, as that would both violate a user's trust and impact a user's battery life.

Violations of the `userVisibleOnly` promise will result in a push subscription being revoked.

A little bit about WebKit

Some of you are interested in the implementation details of Web Push in WebKit.

One goal of the WebKit open source project is to make it easy to deliver a modern browser engine that integrates well with any modern platform.

Many web-facing features are implemented entirely within WebKit, and the maintainers of a given WebKit port do not have

to do any additional work to add support on their platforms.

Occasionally features require relatively deep integration with a platform. That means a WebKit port needs to write a lot of custom code inside WebKit or integrate with platform specific libraries. For example, to support the HTML `<audio>` and `<video>` elements, Apple's port leverages Apple's Core Media framework, whereas the GTK port uses the GStreamer project.

A feature might also require deep enough customization on a per-Application basis that WebKit can't do the work itself.

For example web content might call `window.alert()`. In a general purpose web browser like Safari, the browser wants to control the presentation of the alert itself. But an e-book reader that displays web content might want to suppress alerts altogether.

From WebKit's perspective, supporting Web Push requires deep per-platform and per-application customization.

Web Push in Apple's WebKit port

Apple's WebKit port includes a new daemon called `webpushd`. It is installed as a LaunchAgent in macOS Ventura to support Web Push. This daemon takes push subscription requests from webpages in Safari 16 and turns them into actual push subscriptions with the Apple Push Notification service.

Incoming pushes to the system are delivered to `webpushd`, which then wakes the appropriate application to hand off any pending push messages to a service worker.

The promise of Web Push is that you can reach your users even if they don't have your website open in a browser tab. Because of how we integrated `webpushd` with built-in push support in

macOS Ventura, Safari doesn't even need to be running for a push message to be delivered.

The requirement to display user visible notifications is another platform specific point. Different browsers might implement Notifications API support in different ways. Safari has always supported local notifications by relying on the macOS Notification Center and has made additional changes to handle activating these notifications when Safari is not running.

Integrating Apple Push Notification service's new Web Push support with `webpushd` and supporting notifications while Safari isn't running are both system-level changes, making our implementation require macOS Ventura and later.

More resources

Apple has a few more resources to learn more about Web Push support in Safari 16 on macOS Ventura:

- Read [Sending web push notifications in Safari and other browsers](#)
- Watch [Meet Web Push for Safari](#) (15 minutes)

MDN has some great resources on Web Push. You should start out with [Web Push API Notifications best practices](#).

And of course you can always reference the W3C standards directly:

- [Push API](#)
- [Notifications API](#)
- [ServiceWorker API](#)

Next

Release Notes for Safari Technology Preview 147 with Safari 16 Features

[Learn more](#)

Previously

Web technology sessions at WWDC22

[Learn more](#)

[@webkit@front-end.social](#)

[Site Map](#)

[Privacy Policy](#)

[Licensing WebKit](#)

WebKit and the WebKit logo are trademarks of Apple Inc.